

Sistema de Fábrica Distribuída - Documentação Técnica



Estado Atual do Projeto

Visão Geral

O sistema de fábrica distribuída encontra-se na **fase de infraestrutura base implementada**, com todos os componentes containerizados e comunicando através de uma rede Docker. A arquitetura é composta por 6 serviços principais executando em containers separados, simulando um ambiente industrial real.

Componentes Implementados

Componente	Status	Porta	Funcionalidade
Controlador Central	✓ Funcional	8080	Coordenação geral do sistema
Robô Soldagem	✓ Operacional	-	Processamento de soldagem
Robô Pintura	✓ Operacional	-	Processamento de pintura
Robô Montagem	✓ Operacional	-	Processamento de montagem
Gerenciador Estoque	Não implementado	8081	Controle de inventário
Monitor Sistema	Não implementado	8082	Monitoramento operacional

Tecnologias Utilizadas

- **Java 21** - Linguagem de programação principal
- **Docker & Docker Compose** - Containerização e orquestração
- **Sockets TCP/IP** - Comunicação entre serviços



Escolha: Processos vs Threads

Decisão Arquitetural: Processos Distribuídos

Justificativa da Escolha

1. Isolamento e Robustez

- Cada robô/serviço executa em um **processo separado** (container Docker)
- Falha em um componente não afeta os demais
- Maior estabilidade do sistema como um todo

2. Escalabilidade Horizontal

- Facilita a adição de novos robôs/serviços
- Possibilidade de executar em **múltiplas máquinas físicas**
- Balanceamento de carga natural

3. Manutenção e Deploy

- Atualizações independentes de cada componente
- Facilita debugging e monitoramento
- Deploy incremental possível

4. Simulação de Ambiente Real

- Reflete melhor um ambiente industrial real
- Cada robô físico seria um sistema independente
- Preparação para distribuição geográfica

Processos (Arquitetura geral):

- Cada container = um processo independente
- Comunicação via sockets TCP/IP
- Isolamento total de memória e recursos

Implementação de Sockets

1. Servidor (Controlador Central)

```
public class ControladorCentral {  
    private ServerSocket serverSocket;  
    private List<Socket> robotConnections = new ArrayList<>();  
  
    public void iniciarServidor(int porta) throws IOException {  
        serverSocket = new ServerSocket(porta);  
        System.out.println("🌀 Controlador Central rodando na porta " + porta);  
  
        while (true) {  
            Socket clientSocket = serverSocket.accept();  
            robotConnections.add(clientSocket);  
  
            // Thread para cada robô conectado  
            new Thread(() -> gerenciarRobo(clientSocket)).start();  
        }  
    }  
}
```

```

    }
}

private void gerenciarRobo(Socket robotSocket) {
    try (BufferedReader in = new BufferedReader(
        new InputStreamReader(robotSocket.getInputStream()));
        PrintWriter out = new PrintWriter(
            robotSocket.getOutputStream(), true)) {

        String tipoRobo = in.readLine();
        System.out.println("🤖 Robô " + tipoRobo + " conectado");

        // Lógica de comunicação
        String comando;
        while ((comando = in.readLine()) != null) {
            processarComando(comando, out);
        }
    } catch (IOException e) {
        System.err.println("Erro na comunicação: " + e.getMessage());
    }
}
}

```

2. Cliente (Robôs)

```

public class RoboDistribuido {
    private Socket socket;
    private PrintWriter out;
    private BufferedReader in;

    public void conectarControlador(String host, int porta, String tipoRobo) {
        try {
            socket = new Socket(host, porta);
            out = new PrintWriter(socket.getOutputStream(), true);
            in = new BufferedReader(new InputStreamReader(socket.getInputStream()));

            // Identificar tipo do robô
            out.println(tipoRobo);

            System.out.println("🔗 Robô " + tipoRobo + " conectado ao controlador");

            // Loop de comunicação
            escutarComandos();

        } catch (IOException e) {
            System.err.println("Erro de conexão: " + e.getMessage());
        }
    }
}

```

```

private void escutarComandos() throws IOException {
    String comando;
    while ((comando = in.readLine()) != null) {
        executarComando(comando);
    }
}
}

```

Protocolo de Comunicação

Formato das Mensagens

```

CONNECT:<TIPO_ROBO>      // Conexão inicial
TASK:<TIPO_TASK>:<DADOS>  // Atribuição de tarefa
STATUS:<ROBO_ID>:<STATUS> // Relatório de status
RESULT:<TASK_ID>:<RESULT> // Resultado da tarefa
ERROR:<ERROR_CODE>:<MSG>  // Relatório de erro

```

Exemplo de Fluxo

1. **Robô** → **Controlador**: `CONNECT:SOLDAGEM`
2. **Controlador** → **Robô**: `TASK:SOLDAR_PECA:{"id": 123, "material": "aco"}`
3. **Robô** → **Controlador**: `STATUS:SOLDAGEM:PROCESSANDO`
4. **Robô** → **Controlador**: `RESULT:123:SUCESSO`

Instruções de Execução

Pré-requisitos

- **Docker** instalado (versão 20.10+)
- **Docker Compose** instalado (versão 1.29+)
- **Java 21** (apenas para desenvolvimento)

1. Preparação do Ambiente

```

# Clonar o repositório (se aplicável)
git clone <repository-url>
cd sistema-fabrica-distribuida

```

```

# Verificar estrutura do projeto
ls -la

```

```

# Deve conter: controlador/ robo/ estoque/ monitor/ docker-compose.yml

```

2. Verificar Arquivos de Configuração

Estrutura Necessária:

```
projeto/
├── controlador/
│   ├── Dockerfile
│   └── src/
├── robo/
│   ├── Dockerfile
│   └── src/
├── estoque/
│   ├── Dockerfile
│   └── src/
├── monitor/
│   ├── Dockerfile
│   └── src/
└── docker-compose.yml
```

3. Executar o Sistema

Método 1: Execução com docker (Recomendado)

Pré-requisitos: Docker instalado

Executar o projeto com docker-compose

Na raiz do projeto execute no terminal o seguinte comando para subir todos os componentes e executar o projeto:

docker-compose up

Método 2: Maven (Desenvolvimento)

Pré-requisitos: Java 21 e Maven instalados

Executar cada componente em terminais separados

Terminal 1 - Controlador Central

cd controlador/

mvn clean compile exec:java -Dexec.mainClass="org.example.ControladorCentral"

Terminal 2 - Gerenciador de Estoque

cd estoque/

mvn clean compile exec:java -Dexec.mainClass="org.example.GerenciadorEstoque"

Terminal 3 - Monitor do Sistema

cd monitor/

mvn clean compile exec:java -Dexec.mainClass="org.example.MonitorSistema"

Terminal 4 - Robô Soldagem

```
cd robo/  
mvn clean compile exec:java -Dexec.mainClass="org.example.RoboDistribuido"  
-Dexec.args="SOLDAGEM localhost 8080"
```

Terminal 5 - Robô Pintura

```
cd robo/  
mvn clean compile exec:java -Dexec.mainClass="org.example.RoboDistribuido"  
-Dexec.args="PINTURA localhost 8080"
```

Terminal 6 - Robô Montagem

```
cd robo/  
mvn clean compile exec:java -Dexec.mainClass="org.example.RoboDistribuido" -Dexec.ar
```

Método 3: Java Puro (Compilação Manual)

Pré-requisitos: Java 21 instalado

Executar cada componente em terminais separados

Terminal 1 - Controlador Central

```
cd controlador/  
javac -d target/classes src/main/java/org/example/*.java  
java -cp target/classes org.example.ControladorCentral
```

Terminal 2 - Gerenciador de Estoque

```
cd estoque/  
javac -d target/classes src/main/java/org/example/*.java  
java -cp target/classes org.example.GerenciadorEstoque
```

Terminal 3 - Monitor do Sistema

```
cd monitor/  
javac -d target/classes src/main/java/org/example/*.java  
java -cp target/classes org.example.MonitorSistema
```

Terminal 4 - Robô Soldagem

```
cd robo/  
javac -d target/classes src/main/java/org/example/*.java  
java -cp target/classes org.example.RoboDistribuido SOLDAGEM localhost 8080
```

Terminal 5 - Robô Pintura

```
cd robo/  
javac -d target/classes src/main/java/org/example/*.java  
java -cp target/classes org.example.RoboDistribuido PINTURA localhost 8080
```

Terminal 6 - Robô Montagem

```
cd robo/
```

```
javac -d target/classes src/main/java/org/example/*.java
```

```
java -cp target/classes org.example.RoboDistribuido MONTAGEM localhost 8080
```

8. Logs de Sucesso Esperados

- ✓ monitor-sistema |  Monitor do sistema em execução
- ✓ gerenciador-estoque |  Estoque iniciado
- ✓ controlador-central |  Controlador Central rodando na porta 8080
- ✓ controlador-central |  Hostname: <hostname>
- ✓ controlador-central |  IP: <ip-address>
- ✓ robo-soldagem |  Robô SOLDAGEM conectado ao controlador
- ✓ robo-pintura |  Robô PINTURA conectado ao controlador
- ✓ robo-montagem |  Robô MONTAGEM conectado ao controlador